IFT-4100/7100 : Concepts et applications de la chaîne de blocs

TP3 : Développement d'une DApp sur la blockchain

Date d'échéance : 24 avril 2026

Objectifs

Ce travail pratique vise à vous faire concevoir et livrer une application décentralisée (DApp) complète sur une blockchain compatible EVM : contrat(s) intelligent(s), interface utilisateur, tests et déploiement sur un réseau public de test.

- Acquérir une expérience pratique avec Solidity et la conception de contrats intelligents.
- Comprendre l'architecture d'une DApp (contrat(s) + front-end + portefeuille + réseau).
- Utiliser un framework de développement moderne : Hardhat ou Foundry.
- Déployer et vérifier le bon fonctionnement sur un testnet public (Ethereum ou L2 compatible EVM).
- Interagir avec la chaîne via une librairie front-end (Ethers.js ou viem/wagmi).
- Appliquer des bonnes pratiques de sécurité (contrôles d'accès, prévention de la réentrance, validations d'entrées, etc.).
- Présenter le projet de manière claire (documentation + vidéo de démonstration).

Choix du projet

Vous devez en choisir un parmi les trois thèmes suivants :

1. Art Marketplace – plateforme permettant de créer et vendre des NFT.
2. Fruit Market – marketplace décentralisée pour acheter/vendre des produits.
3. Location d'autos – service de location de voitures décentralisé.

Livrables attendus (obligatoires)

À remettre sur la plateforme du cours :

- Code source (archive ou dépôt) incluant : contrat(s), tests, scripts de déploiement/upgrade, et front-end.
- Un fichier README expliquant : installation, commandes pour compiler/tester/déployer, et la procédure de démonstration.
- Les adresses des contrats déployés, le réseau utilisé, et les liens vers l'explorateur de blocs.
- Une vidéo de démonstration (voir la section « Vidéo »).

Important : ne jamais déposer de clé privée ou de secrets dans votre dépôt. Utilisez un fichier `.env` non versionné.

Contraintes techniques

Framework

Vous devez utiliser l'un des deux :

- Hardhat (JavaScript/TypeScript), ou
- Foundry (Forge/Anvil/Cast).

Réseau de déploiement (deux options acceptées)

Vous pouvez choisir l'une des deux options ci-dessous. Dans tous les cas, indiquez clairement dans le README et la vidéo le réseau choisi, et fournissez les liens *explorateur*.

- **Option A – Ethereum testnet** : Sepolia (recommandé pour le développement de DApps).
- **Option B – L2 EVM testnet** : un testnet L2 compatible EVM est accepté (ex. Base Sepolia, Arbitrum Sepolia, Optimism Sepolia).

Note : le testnet Holesky est déprécié depuis septembre 2025.

Portefeuille et connexion

- Connexion via un portefeuille compatible (MetaMask, WalletConnect, ou équivalent).
- Le front-end doit détecter le réseau, afficher l'adresse connectée, et gérer au minimum les erreurs courantes (réseau non supporté, transaction rejetée, fonds insuffisants).

Sécurité et librairies

Vous devez utiliser une librairie reconnue (ex. OpenZeppelin) pour :

- Contrôles d'accès (Ownable et/ou rôles).
- Protection contre la réentrance (ReentrancyGuard) et bonnes pratiques Checks-Effects-Interactions.
- Validations d'entrées et gestion d'erreurs (require/custom errors).
- Attention particulière aux blocs 'unchecked` et aux conversions de types.

Contrats upgradables (obligatoire)

Votre contrat principal doit être déployé via un schéma proxy reconnu (ex. Transparent Proxy ou UUPS). Vous devez démontrer une mise à niveau v1 → v2 sur le testnet choisi, tout en conservant l'état (storage).

- Livrer une version v1 (fonctionnelle) puis une version v2 (au moins une fonctionnalité ajoutée/modifiée).
- Inclure un script d'upgrade reproductible. Outils recommandés : OpenZeppelin Upgrades (plugin Hardhat) ou équivalent.
- Inclure au moins un test automatisé qui valide : (1) état en v1, (2) upgrade, (3) état conservé + nouvelle fonctionnalité disponible.

Stockage décentralisé (si applicable)

Pour le thème « Art Marketplace », vous devez stocker les médias/métadonnées hors chaîne et référencer ces contenus depuis la chaîne.

- IPFS est recommandé (via un service de pinning ou une solution équivalente).
- Le lien (CID / URI) doit rester accessible au correcteur au moment de l'évaluation.

Démarrage rapide

Option 1 – Hardhat

Exemple de commandes :

```
npm init -y
npm install --save-dev hardhat@hh2
npx hardhat --init
```

Tests et exécution locale :

```
npx hardhat test
npx hardhat node
# Dans un autre terminal :
# npx hardhat run scripts/deploy.ts --network localhost
```

Déploiement sur un testnet (exemple) :

- Configurer le réseau dans hardhat.config.(js|ts) (RPC + clé privée via variables d'environnement).
- Écrire un script de déploiement (scripts/deploy.(js|ts)) et un script d'upgrade (scripts/upgrade.(js|ts)).

```
npx hardhat run scripts/deploy.ts --network sepolia
npx hardhat run scripts/upgrade.ts --network sepolia
```

Option 2 – Foundry

Exemple de commandes :

```
forge init mon-projet
cd mon-projet
forge build
forge test
```

Déploiement via scripts Foundry (exemple) :

```
forge script script/Deploy.s.sol --rpc-url $RPC_URL --private-key
$PRIVATE_KEY --broadcast
forge script script/Upgrade.s.sol --rpc-url $RPC_URL --private-key
$PRIVATE_KEY --broadcast
```

Astuce : gardez vos secrets dans un fichier `.env` et chargez-le (par ex. avec dotenv) selon l'outil choisi.

Exigences par projet

1) Art Marketplace

Contrat intelligent :

- Permettre de créer (mint), lister et vendre des NFT.
- Supporter une vente directe et/ou un mécanisme d'enchères (à préciser dans le README).
- Garder la traçabilité minimale : propriétaire actuel, et historique de propriétaires (ou événements consultables).

Front-end :

- Interface pour téléverser l'œuvre/métadonnées (IPFS recommandé) et publier l'annonce.
- Interface pour parcourir les NFT et acheter/enchérir.
- Connexion wallet (MetaMask, WalletConnect ou équivalent).

2) Fruit Market

Contrat intelligent :

- Permettre aux vendeurs d'ajouter des produits (nom, prix, quantité/stock).
- Permettre l'achat, avec mise à jour de disponibilité et enregistrement des transactions.
- Prévoir des contrôles d'accès pour la modification/suppression d'articles.

Front-end :

- Catalogue des produits, achat, suivi de l'état (disponible/épuisé/commandé).
- Gestion d'erreurs : transaction rejetée, fonds insuffisants, produit indisponible.

3) Location d'autos

Contrat intelligent :

- Permettre de lister des voitures à louer (prix journalier, disponibilité).
- Permettre une réservation/location avec paiement et durée.
- Empêcher les doubles réservations (au minimum pour des périodes qui se chevauchent).
- Optionnel : dépôt de garantie / pénalités / assurance.

Front-end :

- Recherche/parcours des voitures, sélection de période, paiement.
- Affichage clair des disponibilités et des conflits de réservation.

Tests (scénarios recommandés)

Vous devez fournir des tests automatisés. Les tests ci-dessous sont des scénarios recommandés. Minimum attendu : 4 tests pertinents pour votre thème, dont 1 test d'upgrade v1 → v2 (conservation de l'état).

Art Marketplace

1. Déploiement du contrat (adresse non nulle).
2. Mint d'un NFT (propriété correctement attribuée).

3. Mise en vente (prix correctement enregistré).
4. Achat (transfert de propriété et paiement).
5. Historique de possession (ou événements) cohérent.
6. Métadonnées (tokenURI / URI) correctement stockées et récupérables.
7. Gestion des doublons de listing (prévenir ou gérer proprement).

Fruit Market

1. Déploiement du contrat (adresse non nulle).
2. Ajout d'un fruit (nom/prix/stock corrects).
3. Achat (acheteur enregistré, valeur transférée).
4. Disponibilité/stock mis à jour après achat.
5. Évaluation d'un fournisseur (si implémenté) correctement enregistrée.
6. Mise à jour d'une annonce (prix/stock) avec contrôle d'accès.
7. Achat avec fonds insuffisants : la transaction revert et l'état reste cohérent.

Location d'autos

1. Déploiement du contrat (adresse non nulle).
2. Mise en location (paramètres correctement enregistrés).
3. Location (locataire + durée + prix total corrects).
4. Disponibilité mise à jour après location.
5. Restitution : disponibilité rétablie.
6. Calcul de durée/prix exact (cas simple).
7. Conflit de réservation : empêcher une réservation qui se chevauche.

Test obligatoire – Upgrade v1 → v2 (tous thèmes)

Votre suite de tests doit inclure au minimum :

- Déployer v1 via proxy et initialiser l'état.
- Effectuer une action qui modifie l'état (ex. ajouter un produit, mint un NFT, lister une voiture).
- Upgrader vers v2 via le script prévu.
- Vérifier que l'état initial est conservé et que la nouvelle fonctionnalité en v2 est utilisable.

Vidéo (obligatoire)

Votre vidéo (courte et structurée) doit montrer :

- Connexion wallet et interaction front-end → contrat sur le testnet.
- Une transaction complète (création/achat/location, selon le thème).
- La preuve de déploiement : adresse(s) + explorateur de blocs (à l'écran).
- La preuve de l'upgrade v1 → v2 (avant/après).

Critères d’évaluation

La grille suivante s’applique à tous les thèmes :

Critère	Pondération
Rapport vidéo – Démonstration claire et explication technique	15%
Interface – Ergonomie et design du front-end	5%
Qualité Front-End – Cohérence, gestion d’erreurs, lisibilité	10%
Qualité du contrat intelligent – Exactitude et fonctionnalité	35%
Sécurité – Bonnes pratiques (accès, réentrance, validations, etc.)	15%
Déploiement – Déploiement réussi + adresses + liens explorateur	10%
Documentation – README, commentaires, instructions reproductibles	10%

Bonus (facultatif) : vérification du contrat sur l’explorateur, couverture de tests, et/ou CI (GitHub Actions).